

DDBMS Design Issues

Exam-Ready Study Notes

Easy English + বাংলা ব্যাখ্যা সহ

পরীক্ষার আগের রাতে দ্রুত পড়ার জন্য তৈরি নোট।
সহজ ইংরেজি ব্যাখ্যা এবং বাংলা অনুবাদ একসাথে দেওয়া আছে।

Table of Contents / সূচিপত্র

1. Transparency in DDBMS (স্বচ্ছতা)
2. Distribution Transparency & its types
3. Transaction, Performance, DBMS Transparency
4. Distributed Transaction Management (ACID)
5. Transaction Manager vs Transaction Coordinator
6. Concurrency Control in DDBMS
7. Concurrency Control Anomalies
8. Locking Protocols (Read/Write locks, 2PL)
9. Distributed Deadlock Management
10. Deadlock Prevention (Wait-Die / Wound-Wait)
11. Deadlock Detection (Centralized / Distributed)
12. False & Phantom Deadlocks
13. Quick Recall Sheet (Last-Minute Revision)

1. Transparency in DDBMS

What is Transparency?

Original: It refers to the separation of high-level semantics from lower-level implementation issues. It hides implementation details from users.

Easy English: Transparency means "hiding the complicated stuff." The user feels like they are using one simple, normal (centralized) database — even though behind the scenes, data is scattered across many computers/sites.

বাংলা: ট্রান্সপারেন্সি মানে হলো — ব্যবহারকারীর কাছ থেকে জটিল বিষয়গুলো লুকিয়ে রাখা। ব্যবহারকারী মনে করবে সে একটা সাধারণ (centralized) ডেটাবেজ ব্যবহার করছে, কিন্তু আসলে ডেটা অনেকগুলো সাইটে/কম্পিউটারে ছড়ানো থাকে।

Exam Tip: Four main categories — Distribution, Transaction, Performance, DBMS transparency. (Memory trick: D-T-P-D)

Four Main Categories

- **Distribution Transparency** — hides fragmentation, replication, location
- **Transaction Transparency** — hides that a transaction is split across sites
- **Performance Transparency** — system should act as fast as a centralized DBMS
- **DBMS Transparency** — hides that different local DBMS software may be used

বাংলা: চারটি প্রধান ভাগ — ডিস্ট্রিবিউশন ট্রান্সপারেন্সি (ডেটা কোথায় ভাগ হয়ে আছে তা লুকানো), ট্রানজেকশন ট্রান্সপারেন্সি (ট্রানজেকশন একাধিক সাইটে ভাগ হয় তা লুকানো), পারফরম্যান্স ট্রান্সপারেন্সি (গতি যেন কমে না যায়), এবং ডিবিএমএস ট্রান্সপারেন্সি (বিভিন্ন সাইটে আলাদা ডিবিএমএস সফটওয়্যার থাকতে পারে তা লুকানো)।

2. Distribution Transparency — 5 Sub-types

Fragmentation Transparency

Easy English: User doesn't need to know that a table has been broken (fragmented) into pieces. User just asks for data normally, like the whole table is in one place.

বাংলা: ইউজারকে জানতে হয় না যে টেবিলটা ভেঙে (fragment) আলাদা আলাদা টুকরায় রাখা হয়েছে। ইউজার সাধারণভাবেই ডেটা চাইতে পারে।

Location Transparency

Easy English: User must know the fragment's name, but NOT where (which site/computer) it is physically stored.

বাংলা: ইউজার ফ্র্যাগমেন্টের নাম জানবে, কিন্তু সেটা কোন সাইটে/কোন কম্পিউটারে আছে তা জানার দরকার নেই।

Exam Tip: Common confusion — Fragmentation vs Location Transparency. Fragmentation = don't need fragment name. Location = know fragment name, don't need location.

Replication Transparency

Easy English: User doesn't know that copies (replicas) of the same data exist at different sites for backup/speed.

বাংলা: একই ডেটার একাধিক কপি (replica) বিভিন্ন সাইটে রাখা থাকতে পারে — ইউজার এটা জানে না।

Local Mapping Transparency (opposite/lowest transparency)

Easy English: This is when there is LESS hiding — user must know BOTH the fragment name AND its exact location to access data.

বাংলা: এখানে ইউজারকে ফ্র্যাগমেন্টের নাম এবং তার লোকেশন — দুটোই জানতে হয়। এটা সবচেয়ে কম স্বচ্ছতা।

Naming Transparency

Easy English: Every object (table, fragment, replica) needs a unique name across the whole system so two sites don't create the same name by mistake. A central name server can manage this. User uses "alias" (nickname) names instead of the real names.

বাংলা: প্রতিটা অবজেক্টের (টেবিল, ফ্র্যাগমেন্ট, রিপ্লিকা) একটা ইউনিক নাম থাকতে হবে যাতে দুই সাইট একই নাম না বানায়। একটা "central name server" এটা নিশ্চিত করতে পারে। ইউজার আসল নামের বদলে "alias" (ডাকনাম) ব্যবহার করে।

3. Transaction, Performance & DBMS Transparency

Transaction Transparency

Easy English: A big transaction gets divided into small "sub-transactions" running at different sites. Transaction transparency guarantees: the WHOLE transaction succeeds only if ALL sub-transactions succeed. (All-or-nothing rule across sites.)

বাংলা: একটা বড় ট্রানজেকশন ছোট ছোট "সাব-ট্রানজেকশনে" ভাগ হয়ে বিভিন্ন সাইটে চলে। পুরো ট্রানজেকশন তখনই সফল হবে যখন সব সাব-ট্রানজেকশন সফল হবে — নাহলে কোনোটাই হবে না।

Performance Transparency

Easy English: Even though data is spread across many sites, the system should feel just as fast as a normal single-computer database. No slowdown because of being "distributed."

বাংলা: ডেটা বহু সাইটে ছড়ানো থাকলেও সিস্টেমের গতি যেন সাধারণ (centralized) ডেটাবেজের মতোই দ্রুত থাকে।

DBMS Transparency (a.k.a. Heterogeneity Transparency)

Easy English: Different sites might use different DBMS software (like one site uses MySQL, another uses Oracle). This transparency hides that difference and makes everything look like one unified system. Only applies to "heterogeneous" DDBMS. It is the HARDEST transparency to achieve.

বাংলা: বিভিন্ন সাইটে ভিন্ন ভিন্ন ডিবিএমএস সফটওয়্যার থাকতে পারে (যেমন একটাতে MySQL, আরেকটাতে Oracle)। এই ট্রান্সপারেন্সি সেই পার্থক্য লুকিয়ে সব কিছু এক সিস্টেমের মতো দেখায়। এটা বাস্তবায়ন করা সবচেয়ে কঠিন।

Exam Tip: DBMS transparency = heterogeneity transparency. Remember this synonym — commonly asked.

4. Distributed Transaction Management (ACID)

What is a Transaction?

Easy English: A transaction is a group of operations (like read/write) that together do ONE job, and move the database from one correct (consistent) state to another correct state.

বাংলা: ট্রানজেকশন হলো কয়েকটা অপারেশনের (read/write) একটা গ্রুপ, যেগুলো একসাথে একটা নির্দিষ্ট কাজ সম্পন্ন করে এবং ডেটাবেজকে এক সঠিক অবস্থা থেকে আরেক সঠিক অবস্থায় নিয়ে যায়।

ACID Properties (Very Important — asked almost every exam)

Letter	Property	Easy Meaning
A	Atomicity	All-or-nothing: either all changes happen, or none happen
C	Consistency	Database must go from one valid/correct state to another valid state
I	Isolation	Transactions don't interfere with each other; one can't see another's unfinished work
D	Durability	Once committed, changes are permanent — even if system crashes after

বাংলা:

Atomicity (A): হয় সব কাজ হবে, নাহলে কিছুই হবে না (all or nothing)।

Consistency (C): ট্রানজেকশনের পর ডেটাবেজ অবশ্যই একটা সঠিক অবস্থায় থাকতে হবে।

Isolation (I): একটা ট্রানজেকশন আরেকটার অসম্পূর্ণ কাজ দেখতে পারবে না, সবাই স্বাধীনভাবে চলে।

Durability (D): কমিট হয়ে গেলে পরিবর্তনগুলো স্থায়ী থাকবে, সিস্টেম ক্র্যাশ করলেও হারাবে না।

Exam Tip: Write "ACID = Atomicity, Consistency, Isolation, Durability" first — examiners give marks just for stating this correctly, then explain each in 1-2 lines.

Local vs Global Transactions

Easy English:

- **Local transaction** — only touches data at ONE site.
- **Global transaction** — touches data at MULTIPLE sites (harder to manage).

বাংলা: **Local transaction** শুধু একটা সাইটের ডেটা নিয়ে কাজ করে। **Global transaction** একাধিক সাইটের ডেটা নিয়ে কাজ করে — এটা ম্যানেজ করা কঠিন।

5. Transaction Manager (TM) vs Transaction Coordinator (TC)

Model: Two Sub-modules at Each Site

Diagram idea: at every site, there is a TM box and a TC box; TCs at different sites talk to each other to coordinate global transactions.

Module	Job (Easy English)
Transaction Manager (TM)	Manages transactions that touch data at ITS OWN site. Keeps ACID properties for local execution. Maintains logs for recovery. Works with concurrency control.
Transaction Coordinator (TC)	Coordinates BOTH local and global transactions started at that site. Breaks a global transaction into sub-transactions and sends them to the right sites. Also decides final commit or abort for everyone.

বাংলা:

Transaction Manager (TM): নিজের সাইটে চলা ট্রানজেকশনগুলো ম্যানেজ করে, ACID নিশ্চিত করে, রিকভারির জন্য লগ রাখে।

Transaction Coordinator (TC): লোকাল ও গ্লোবাল দুই ধরনের ট্রানজেকশনই কো-অর্ডিনেট করে। বড় ট্রানজেকশনকে ছোট সাব-ট্রানজেকশনে ভেঙে সঠিক সাইটে পাঠায়। শেষে সবাইকে commit বা abort করার সিদ্ধান্ত জানায়।

Exam Tip: Common question: "Differentiate TM and TC." Key line: TM handles single-site execution; TC handles coordination across sites.

6. Concurrency Control in DDBMS

What is Concurrency Control?

Easy English: Many transactions try to use the same data at the same time, from different sites. Concurrency control makes sure they don't mess up each other's work. It's harder in DDBMS because one site doesn't instantly know what's happening at another site, plus data may be replicated.

বাংলা: অনেক ট্রানজেকশন একই সময়ে একই ডেটা ব্যবহার করতে চাইতে পারে, বিভিন্ন সাইট থেকে। কনকারেন্সি কন্ট্রোল নিশ্চিত করে তারা একে অপরের কাজ নষ্ট না করে। DDBMS-এ এটা কঠিন কারণ এক সাইট সাথে সাথে আরেক সাইটের খবর জানে না, আর ডেটা রিপ্লিকেটেডও হতে পারে।

Objectives (Goals) of Distributed Concurrency Control

- Must survive site/communication failures
- Should allow max parallel execution
- Low overhead (simple, not heavy on resources)
- Work well despite network delay
- Few restrictions on transaction structure
- Must keep data consistent

বাংলা: সাইট বা যোগাযোগ ব্যর্থ হলেও কাজ চালাতে হবে; সর্বোচ্চ প্যারালাল এক্সিকিউশন দিতে হবে; ওভারহেড কম রাখতে হবে; নেটওয়ার্ক ডিলে সত্ত্বেও ভালো পারফর্ম করতে হবে; ডেটার কনসিস্টেন্সি বজায় রাখতে হবে।

7. Concurrency Control Anomalies (Problems Without Control)

Five Key Anomalies

Anomaly	Easy Meaning
Lost Update	One transaction's completed update gets overwritten/lost because another transaction updates the same data after
Uncommitted Dependency (Dirty Read)	A transaction reads data that another transaction wrote but hasn't committed yet — then that other transaction aborts, so the read data was never really valid
Inconsistent Analysis	A transaction reads several values, but another transaction changes some of them in between — so the first transaction sees a mixed-up, inconsistent picture
Phantom Read	A transaction reads rows matching a condition; another transaction inserts a NEW row matching that same condition; if the first transaction re-reads, it sees an extra "phantom" row
Multiple-copy Consistency Problem	When data is replicated, updating one copy but not the others makes the database inconsistent

বাংলা:

Lost Update: একটা ট্রানজেকশনের করা আপডেট আরেকটা ট্রানজেকশন এসে মুছে/ওভাররাইট করে দেয়।

Uncommitted Dependency: এক ট্রানজেকশন আরেকটার অসম্পূর্ণ (commit না হওয়া) ডেটা পড়ে ফেলে, পরে সেটা abort হয়ে যায়।

Inconsistent Analysis: একটা ট্রানজেকশন কয়েকটা ভ্যালু পড়ছে, মাঝখানে আরেকটা ট্রানজেকশন কিছু ভ্যালু পাল্টে ফেলে — ফলে অসামঞ্জস্যপূর্ণ ফলাফল পাওয়া যায়।

Phantom Read: একটা শর্ত অনুযায়ী ডেটা পড়ার পর আরেকটা ট্রানজেকশন নতুন সারি (row) যোগ করে যা সেই শর্তে মিলে যায়।

Multiple-copy Consistency: ডেটা রিপ্লিকেটেড থাকলে একটা কপি আপডেট হলে বাকি কপিও আপডেট না হলে ডেটাবেজ অসামঞ্জস্যপূর্ণ হয়ে যায়।

Exam Tip: These 5 anomalies are a favorite short-question topic. Practice writing one line + one small example for each.

8. Locking-Based Concurrency Control & 2PL

What is a Lock?

Easy English: A lock tells the DBMS "this data is being used right now." Before touching any data, a transaction must get a lock on it. This stops other transactions from changing that data at the same time.

বাংলা: লক DBMS-কে জানায় "এই ডেটা এখন ব্যবহার হচ্ছে।" কোনো ডেটা ব্যবহারের আগে ট্রানজেকশনকে লক নিতে হয়, যাতে অন্য ট্রানজেকশন একই সময়ে সেটা পরিবর্তন করতে না পারে।

Read Lock (Shared) vs Write Lock (Exclusive)

Lock Type	Also Called	Meaning
Read Lock	Shared Lock	Can only READ. Other transactions CAN also get read locks at same time (read-read = no conflict)
Write Lock	Exclusive Lock	Can READ and WRITE/update. NO other transaction can read or write that data (read-write and write-write = conflict)

বাংলা: **Read Lock (Shared):** শুধু পড়া যায়, একাধিক ট্রানজেকশন একসাথে read lock নিতে পারে। **Write Lock (Exclusive):** পড়া ও লেখা দুটোই করা যায়, কিন্তু তখন অন্য কেউ সেই ডেটা পড়তে বা লিখতে পারবে না।

Lock Manager — Basic Operation

Easy English: The Lock Manager controls all locks. When a transaction wants a lock, the Transaction Manager asks the Lock Manager. If the data is already locked in a conflicting way, the transaction must WAIT. Otherwise, the lock is given.

বাংলা: Lock Manager সব লক নিয়ন্ত্রণ করে। ট্রানজেকশন লক চাইলে Transaction Manager এই অনুরোধ Lock Manager-কে পাঠায়। যদি ডেটা আগে থেকেই কনফ্লিক্টিং লকে থাকে, তাহলে ট্রানজেকশনকে অপেক্ষা করতে হয়।

Upgrade / Downgrade of Lock

Easy English: **Upgrade** = read lock → write lock (starts by only reading, later needs to write too). **Downgrade** = write lock → read lock (had write access, later only needs read, releasing exclusivity for others).

বাংলা: **Upgrade** = রিড লক থেকে রাইট লকে পরিবর্তন। **Downgrade** = রাইট লক থেকে রিড লকে পরিবর্তন।

Two-Phase Locking (2PL) Protocol — VERY IMPORTANT

Easy English: Rule: once a transaction releases ANY lock, it can never acquire a NEW lock again. Life of transaction has 2 phases:

- **Growing Phase** — transaction can only GET locks (acquire), cannot release any
- **Shrinking Phase** — transaction can only RELEASE locks, cannot get any new ones

The moment the transaction releases its first lock, growing phase ends and shrinking phase begins.

বাংলা: নিয়ম: একটা ট্রানজেকশন যদি একবার কোনো লক ছেড়ে দেয়, তাহলে সে আর নতুন কোনো লক নিতে পারবে না। ট্রানজেকশনের জীবনে দুইটা ধাপ থাকে —

Growing Phase: শুধু লক নেওয়া যায়, ছাড়া যায় না।

Shrinking Phase: শুধু লক ছাড়া যায়, নতুন লক নেওয়া যায় না।

যখনই প্রথম লক ছাড়া হয়, তখনই Growing Phase শেষ হয়ে Shrinking Phase শুরু হয়।

Exam Tip: 2PL is one of the most-asked topics. Draw the growing/shrinking phase diagram (a simple upward-then-downward line graph of "number of locks held" vs "time") if you can — it earns extra marks.

9. Distributed Deadlock Management

What is a Deadlock?

Easy English: A deadlock happens when transactions wait for each other FOREVER — like Transaction A waits for B, and B waits for A, so neither can proceed. Locking-based (and some timestamp-based) methods can create this.

বাংলা: ডেডলক তখন হয় যখন ট্রানজেকশনগুলো একে অপরের জন্য চিরকাল অপেক্ষা করতে থাকে — যেমন A অপেক্ষা করছে B-এর জন্য, আর B অপেক্ষা করছে A-এর জন্য। ফলে কেউই এগোতে পারে না।

Wait-For Graph (WFG)

Easy English: A picture showing which transaction is waiting for which. Nodes = transactions. An arrow from $T_i \rightarrow T_j$ means "Ti is waiting for a lock held by Tj." **Rule: If the graph has a CYCLE, there IS a deadlock.**

বাংলা: এটা একটা চিত্র যেখানে দেখানো হয় কোন ট্রানজেকশন কার জন্য অপেক্ষা করছে। T_i থেকে T_j -তে তীর মানে "Ti, T_j -এর ধরে রাখা লকের জন্য অপেক্ষা করছে।" **নিয়ম: গ্রাফে যদি সাইকেল (cycle) থাকে তাহলে ডেডলক আছে।**

Exam Tip: Memorize: "Deadlock exists if and only if the Wait-For Graph contains a cycle." This exact line is often asked.

LWFG vs GWFG

Easy English:

- **LWFG** (Local Wait-For Graph) — drawn at ONE site only, using local transactions/data
- **GWFG** (Global Wait-For Graph) — combines ALL the LWFGs from every site together

Important: even if NO site shows a cycle locally, combining them into GWFG might reveal a hidden global deadlock!

বাংলা: **LWFG** শুধু একটা সাইটের জন্য বানানো হয়। **GWFG** সব সাইটের LWFG একসাথে মিলিয়ে বানানো হয়। গুরুত্বপূর্ণ: প্রতিটা সাইটে আলাদাভাবে কোনো সাইকেল না থাকলেও, GWFG বানালে একটা লুকানো গ্লোবাল ডেডলক ধরা পড়তে পারে!

Three General Techniques to Handle Deadlock

- **Deadlock Prevention** — design the system so deadlock CANNOT happen at all
- **Deadlock Avoidance** — check before allowing a wait, to avoid risk
- **Deadlock Detection & Recovery** — let deadlock happen, then detect it and fix it (most popular in practice)

বাংলা: **Prevention** — সিস্টেম এমনভাবে বানানো যাতে ডেডলক হতেই না পারে। **Avoidance** — অপেক্ষা করার আগে চেক করে ঝুঁকি এড়ানো। **Detection & Recovery** — ডেডলক হতে দেওয়া, পরে সেটা শনাক্ত করে সমাধান করা (এটাই বাস্তবে সবচেয়ে বেশি ব্যবহৃত)।

10. Deadlock Prevention: Wait-Die & Wound-Wait

Basic Idea: Priority Using Timestamps

Easy English: Each transaction gets a unique timestamp when it starts (older transaction = smaller/earlier timestamp = higher priority). We use timestamps (instead of simple priority numbers) because plain priority can cause a transaction to be restarted again and again forever (cyclic restart). Timestamps solve this because older transactions always keep the same "birth time," so eventually they win.

বাংলা: প্রতিটা ট্রানজেকশন শুরু হওয়ার সময় একটা ইউনিক টাইমস্ট্যাম্প পায় (পুরনো ট্রানজেকশনের টাইমস্ট্যাম্প ছোট/আগের হয়)। শুধু প্রায়োরিটি নম্বর ব্যবহার করলে একটা ট্রানজেকশন বারবার restart হতে থাকতে পারে (cyclic restart) — টাইমস্ট্যাম্প ব্যবহার করলে এই সমস্যা হয় না, কারণ পুরনো ট্রানজেকশনের টাইমস্ট্যাম্প কখনো পরিবর্তন হয় না।

Wait-Die (Non-preemptive)

Easy English: Rule: OLDER transaction is allowed to WAIT. YOUNGER transaction gets ABORTED (killed) and restarted with the SAME old timestamp.

- If T_i (requesting) is OLDER than T_j (holding lock) → T_i WAITS
- If T_i is YOUNGER than T_j → T_i DIES (aborts) and restarts

Non-preemptive means: the transaction that already HOLDS the lock (T_j) is never forced to give it up.

বাংলা: নিয়ম: পুরনো ট্রানজেকশন অপেক্ষা করতে পারে। নতুন (কম বয়সী) ট্রানজেকশন abort হয়ে যায় এবং একই পুরনো টাইমস্ট্যাম্প নিয়ে আবার শুরু হয়।

T_i পুরনো হলে → T_i অপেক্ষা করে।

T_i নতুন হলে → T_i "die" করে (abort) এবং আবার শুরু হয়।

Non-preemptive মানে: যে ট্রানজেকশন লক ধরে রেখেছে (T_j), তাকে জোর করে লক ছাড়তে হয় না।

Wound-Wait (Preemptive)

Easy English: Rule (opposite direction): YOUNGER transaction is allowed to WAIT for an older one. If a YOUNGER transaction holds the lock and an OLDER transaction requests it, the younger one gets ABORTED ("wounded") and the lock is given to the older transaction immediately.

- If T_i (requesting) is OLDER than T_j (holding lock) → T_j is ABORTED, T_i gets the lock now
- If T_i is YOUNGER than T_j → T_i WAITS

Preemptive means: an older transaction can force a younger one to give up its lock right away.

বাংলা: নিয়ম (উল্টা দিক): নতুন (কম বয়সী) ট্রানজেকশন পুরনোটোর জন্য অপেক্ষা করতে পারে। যদি নতুন ট্রানজেকশন লক ধরে রাখে আর পুরনো ট্রানজেকশন সেটা চায়, তাহলে নতুন ট্রানজেকশনটা abort ("wounded") হয়ে যায় এবং লকটা সাথে সাথে পুরনো ট্রানজেকশনকে দেওয়া হয়।

Preemptive মানে: পুরনো ট্রানজেকশন জোর করে নতুন ট্রানজেকশনের কাছ থেকে লক কেড়ে নিতে পারে।

Exam Tip: Easy memory trick — "Wait-Die: OLD waits, YOUNG dies." "Wound-Wait: YOUNG waits, OLD wounds (kills) young." Both prevent cyclic restart because the direction of waiting always flows one way (old→young or young→old), never both ways.

11. Deadlock Detection Methods

Three Techniques

- **Centralized Deadlock Detection** — one single site (DDC) does everything
- **Hierarchical Deadlock Detection** — detection responsibility arranged in a tree/hierarchy of sites
- **Distributed Deadlock Detection** — every site shares equal responsibility, no central authority

বাংলা: **Centralized** — একটা মাত্র সাইট (DDC) সব কাজ করে। **Hierarchical** — সাইটগুলো একটা ট্রি/হায়ারার্কি আকারে দায়িত্ব ভাগ করে। **Distributed** — প্রতিটা সাইটের সমান দায়িত্ব, কোনো কেন্দ্রীয় কর্তৃপক্ষ নেই।

Centralized Deadlock Detection (DDC method)

Easy English: One site becomes the "Deadlock Detection Coordinator" (DDC). Every site's lock manager periodically sends its LWFG to the DDC. The DDC merges them into one GWFG and looks for cycles. If found, DDC picks a "victim" transaction to abort/rollback and tells the right lock managers.

বাংলা: একটা সাইট "Deadlock Detection Coordinator (DDC)" হয়। প্রতিটা সাইটের lock manager নিয়মিত তাদের LWFG DDC-কে পাঠায়। DDC এগুলো মিলিয়ে GWFG বানায় এবং সাইকেল খোঁজে। সাইকেল পেলে একটা "victim" ট্রানজেকশন বেছে নিয়ে abort/rollback করায়।

Exam Tip — Drawbacks of Centralized method (very commonly asked):

- Less reliable — if the central (DDC) site fails, detection stops completely
- High communication cost — every site keeps sending LWFGs to one place
- False deadlocks can be detected (unnecessary rollbacks)

বাংলা (Drawbacks): কেন্দ্রীয় সাইট ব্যর্থ হলে পুরো ডিটেকশন বন্ধ হয়ে যায়; যোগাযোগের খরচ বেশি; মাঝে মাঝে ভুলভাবে ডেডলক শনাক্ত হয়ে যায় (false deadlock)।

Distributed Deadlock Detection (Obermarck's Method, 1982)

Easy English: Each site builds its own LWFG. An extra "external node" called **Tex** is added to represent transactions from OTHER (remote) sites.

- Edge $T_i \rightarrow \text{Tex}$: T_i is waiting for something held at a remote site
- Edge $\text{Tex} \rightarrow T_i$: some remote transaction is waiting for something T_i holds

Rule for detection:

- Cycle NOT involving $\text{Tex} \rightarrow$ deadlock is LOCAL (can handle locally)
- Cycle involving $\text{Tex} \rightarrow$ POSSIBLE global deadlock — need to merge LWFGs from other sites to confirm

To avoid sending LWFGs everywhere, sites use timestamps: Site S_i sends its LWFG to Site S_k only if a transaction at S_k is waiting for a transaction at S_i AND the S_i transaction is older (smaller timestamp). This way, LWFGs get merged step-by-step until either a real cycle (without Tex) shows up (= deadlock) or the whole GWFG is built with no cycle (= no deadlock).

বাংলা: প্রতিটা সাইট নিজের LWFG বানায়। একটা অতিরিক্ত "external node" Tex যোগ করা হয় যা রিমোট সাইটের ট্রানজেকশনগুলোকে প্রতিনিধিত্ব করে।

$T_i \rightarrow \text{Tex}$: T_i কোনো রিমোট সাইটে ধরে রাখা কিছু জন্য অপেক্ষা করছে।

$\text{Tex} \rightarrow T_i$: কোনো রিমোট ট্রানজেকশন T_i -এর ধরে রাখা কিছু জন্য অপেক্ষা করছে।

যদি Tex ছাড়া সাইকেল থাকে $\rightarrow$ লোকাল ডেডলক। যদি Tex সহ সাইকেল থাকে $\rightarrow$ গ্লোবাল ডেডলকের সম্ভাবনা আছে, নিশ্চিত করতে অন্য সাইটের LWFG মিলাতে হবে।

12. False Deadlocks & Phantom Deadlocks

False Deadlock

Easy English: Because messages take time to travel between sites, the deadlock detector might see OLD information. Example: detector hears "Ti is waiting for Tj." Later, Tj actually releases that item and starts waiting for something Ti holds. If the detector gets the SECOND message before the "Ti is free now" message, it looks like a cycle (deadlock) — but really there isn't one. This is caused purely by MESSAGE DELAY.

বাংলা: সাইটগুলোর মধ্যে মেসেজ পৌঁছাতে সময় লাগে, তাই ডেডলক ডিটেক্টর কখনো কখনো পুরনো তথ্য দেখে ভুল বোঝে। যেমন: ডিটেক্টর জানলো "Ti, Tj-এর জন্য অপেক্ষা করছে।" পরে Tj আসলে সেই ডেটা ছেড়ে দিয়ে Ti-এর ধরে রাখা কিছু জন্য অপেক্ষা শুরু করলো। যদি ডিটেক্টর "Ti মুক্ত হয়ে গেছে" এই খবর পাওয়ার আগেই দ্বিতীয় খবরটা পায়, তাহলে একটা সাইকেল (ডেডলক) মনে হবে — কিন্তু বাস্তবে ডেডলক নেই। এটা শুধু মেসেজ দেরি হওয়ার কারণে ঘটে।

Phantom Deadlock

Easy English: A transaction Ti that is blocking someone else might get restarted for some OTHER reason (not related to deadlock). But if the "Ti restarted" message hasn't reached the deadlock detector yet, the detector may still see Ti in the wait-for graph and wrongly detect a deadlock (a "phantom" / fake one). This can cause the system to unnecessarily abort a different, innocent transaction.

বাংলা: একটা ট্রানজেকশন Ti যেটা অন্য কাউকে আটকে রেখেছে, সেটা অন্য কোনো কারণে (ডেডলকের সাথে সম্পর্কহীন) restart হয়ে যেতে পারে। কিন্তু "Ti restart হয়েছে" এই খবর ডেডলক ডিটেক্টরের কাছে না পৌঁছালে, ডিটেক্টর ভুলভাবে wait-for graph-এ Ti-কে দেখে একটা ভুয়া (phantom) ডেডলক শনাক্ত করতে পারে। এতে অকারণে অন্য একটা নির্দোষ ট্রানজেকশন abort হয়ে যেতে পারে।

Exam Tip: Difference question often asked: "False deadlock vs Phantom deadlock."

False deadlock = caused by delayed WAIT messages making a fake cycle appear.

Phantom deadlock = caused by a delayed RESTART/release message, so an already-resolved transaction still appears stuck in the graph.

13. Quick Recall Sheet — Last Minute Revision / শেষ মুহূর্তের রিভিশন

Transparency Types

Distribution (Fragmentation, Location, Replication, Local Mapping, Naming) — Transaction — Performance — DBMS (Heterogeneity)

ACID

Atomicity (all-or-nothing) — Consistency (valid state to valid state) — Isolation (no interference) — Durability (permanent after commit)

TM vs TC

TM = manages local execution + logs + local concurrency. TC = coordinates local & global transactions, splits transaction, decides commit/abort.

Locks

Read/Shared lock = read only, multiple allowed. Write/Exclusive lock = read+write, only one allowed.

2PL

Growing phase = only acquire locks. Shrinking phase = only release locks. Cannot acquire after first release.

Anomalies

Lost Update, Uncommitted Dependency, Inconsistent Analysis, Phantom Read, Multiple-copy Consistency Problem

Deadlock = Cycle in Wait-For Graph

LWFG = one site. GWFG = merged from all sites. Deadlock exists IFF wait-for graph has a cycle.

3 Deadlock Handling Techniques

Prevention (impossible by design) — Avoidance (check before waiting) — Detection & Recovery (most popular)

Wait-Die vs Wound-Wait

Wait-Die (non-preemptive): old waits, young dies.

Wound-Wait (preemptive): young waits, old wounds (kills) young.

Deadlock Detection Techniques

Centralized (1 DDC site — simple but unreliable, costly, false deadlocks) — Hierarchical — Distributed (Obermarck's method using Tex external node)

False vs Phantom Deadlock

False = delayed wait message creates a fake cycle. Phantom = delayed restart/release message keeps a resolved transaction looking stuck.

বাংলা কুইক রিভিশন

ট্রান্সপারেন্সি: Distribution, Transaction, Performance, DBMS।

ACID: Atomicity, Consistency, Isolation, Durability।

TM = লোকাল ম্যানেজমেন্ট; TC = কো-অর্ডিনেশন।

Read Lock = শুধু পড়া, একাধিক জন নিতে পারে। Write Lock = পড়া+লেখা, শুধু একজন।

2PL: Growing (শুধু নেওয়া) → Shrinking (শুধু ছাড়া)।

৫টা অ্যানোমালি: Lost Update, Uncommitted Dependency, Inconsistent Analysis, Phantom Read, Multiple-copy Consistency।

ডেডলক = Wait-For Graph-এ সাইকেল থাকলে।

৩টা টেকনিক: Prevention, Avoidance, Detection & Recovery।

Wait-Die: পুরনো অপেক্ষা করে, নতুন মারা যায়। Wound-Wait: নতুন অপেক্ষা করে, পুরনো নতুনকে মারে।

Detection ৩ প্রকার: Centralized, Hierarchical, Distributed।